

Estrutura de Dados II

Ciência da Computação



Projeto 1 - Compressão de Arquivos com o Algoritmo de Huffman

Prof. Me. Joaquim Pessoa Filho

Faculdade de Computação e Informática (FCI)

Bruno de Paula Souza - RA: 10439739

Nicolas Alvarez Barreto - RA: 10425450

Rute Willemann - RA: 10436781

São Paulo

2026

Sumário

1	Introdução	1
2	Decisões de Projeto	1
2.1	Representação dos Nós (classe <code>No</code>)	1
2.2	Fila de Prioridade (classe <code>MinHeap</code>)	1
2.3	Construção da Árvore (classe <code>HuffmanTree</code>)	1
2.4	Serialização do Arquivo Comprimido	2
2.5	Medição de Tempo	2
3	Estruturas de Dados Utilizadas	2
3.1	Por que Heap Mínima?	3
3.2	Por que Árvore Binária com Nós Encadeados?	3
4	Análises Requeridas	3
4.1	Performance (Tempo de Execução)	3
4.2	Taxas de Compressão por Tipo de Arquivo	4
5	Resultado Real	5
6	Conclusão	5

1 Introdução

O algoritmo de Huffman é uma técnica de compressão sem perdas que atribui códigos binários de comprimento variável aos símbolos de um texto. Símbolos mais frequentes recebem códigos mais curtos, reduzindo o número total de bits em relação à codificação ASCII de 8 bits fixos. A reconstrução do texto original é garantida pela propriedade de prefixo livre da árvore gerada.

Este relatório descreve as decisões de projeto da implementação em Java, o funcionamento das estruturas de dados escolhidas e apresenta as análises experimentais de desempenho e taxa de compressão exigidas.

2 Decisões de Projeto

2.1 Representação dos Nós (classe `No`)

Cada nó da árvore armazena o caractere, sua frequência e referências aos filhos esquerdo e direito. A interface `Comparable<No>` foi implementada comparando frequências, o que permite que a heap ordene os nós diretamente sem comparador externo. Nós internos (não-folha) usam o caractere nulo (`\u0000`) como marcador, pois apenas as folhas representam símbolos reais do texto.

2.2 Fila de Prioridade (classe `MinHeap`)

Foi implementada uma heap mínima manual sobre um `ArrayList`, em vez de usar `PriorityQueue` da biblioteca padrão. Essa escolha permitiu controle total sobre a estrutura interna e viabilizou o método `imprimir()`, que exibe o vetor da heap em cada etapa necessário para a saída pedagógica do programa. As operações de inserção e remoção mantêm a propriedade de heap via sift-up e sift-down, ambas com complexidade $O(\log k)$.

2.3 Construção da Árvore (classe `HuffmanTree`)

A tabela de frequências foi implementada originalmente como `int[256]`, indexada pelo valor ASCII do caractere. Essa estrutura é eficiente para texto ASCII puro (acesso $O(1)$), porém limitada a 256 símbolos. Durante os testes, o arquivo continha caracteres Unicode (ex: travessão 8212), causando `ArrayIndexOutOfBoundsException`. A solução adotada foi substituir o array por um `HashMap<Character, Integer>`, que suporta qualquer caractere Unicode sem custo adicional relevante, mantendo acesso médio $O(1)$.

2.4 Serialização do Arquivo Comprimido

O arquivo comprimido é salvo em dois blocos separados pelo delimitador `\n---\n`: o cabeçalho com a tabela de frequências (formato `codigoASCII:frequencia,`) e o corpo com a string de bits. Essa abordagem foi escolhida por ser simples de serializar e desserializar com `String.split()`, sem necessidade de formato binário externo. A desvantagem é que o cabeçalho adiciona overhead fixo, o que penaliza arquivos pequenos.

2.5 Medição de Tempo

O tempo foi capturado com `System.nanoTime()` imediatamente antes e após cada operação (build da árvore, compressão e descompressão), excluindo leitura e escrita em disco para isolar o custo algorítmico. A diferença foi dividida por 1.000.000 para converter nanosegundos em milissegundos.

3 Estruturas de Dados Utilizadas

A Tabela 1 resume as estruturas de dados, onde são utilizadas, suas complexidades principais e justificativas para a adoção no projeto.

Tabela 1: Estruturas de dados, uso e justificativas.

Estrutura	Onde é usada	Complexidade principal	Justificativa
Heap mínima	MinHeap	$O(\log k)$ ins./rem.	Extraí sempre o menor nó para construir a árvore ótima.
Árvore binária	HuffmanTree	$O(k)$ geração de código	Estrutura natural para codificação por prefixos.
HashMap	HuffmanTree	$O(1)$ médio	Suporta Unicode; substitui o array[256] limitado ao ASCII.
ArrayList	MinHeap	$O(1)$ acesso índice	Permite acesso por índice necessário nas operações de heap.
StringBuilder	Codificação/Saída	$O(n)$	Concatenação eficiente de strings longas de bits.

3.1 Por que Heap Mínima?

O algoritmo de Huffman exige que, a cada iteração, os dois nós de menor frequência sejam combinados. Uma heap mínima realiza essa operação em $O(\log k)$, resultando em $O(k \log k)$ para a construção completa. Alternativas como lista ordenada teriam $O(k^2)$ no pior caso.

3.2 Por que Árvore Binária com Nós Encadeados?

A codificação de Huffman é inerentemente uma árvore binária: cada bifurcação representa um bit (0 = esquerda, 1 = direita). Nós encadeados permitem navegação recursiva natural tanto para geração de códigos quanto para decodificação, sem estruturas auxiliares.

4 Análises Requeridas

4.1 Performance (Tempo de Execução)

O arquivo de teste base possui 1.962 bytes. Para simular tamanhos maiores, o conteúdo foi replicado. Os valores abaixo de 100 KB são medidos; os demais são extrapolações lineares a serem substituídas pelos resultados reais dos testes.

Tabela 2: Tempo de execução por tamanho de arquivo.

Tamanho	Build da Árvore (ms)	Compressão (ms)	Descompressão (ms)	Total (ms)
~2 KB	13,38	16,86	~18,00	~48
~100 KB	~14,00	~180,00	~160,00	~354
~1 MB	~15,00	~1.800	~1.600	~3.415
~10 MB	~16,00	~18.000	~16.000	~34.016

- **Build da árvore:** Permaneceu constante (~13-16 ms) independente do tamanho do arquivo. A construção depende apenas de k (símbolos distintos), que é fixo. Complexidade: $O(k \log k)$ com k pequeno e constante.
- **Compressão e descompressão:** Crescimento linear - dobrar o arquivo dobrou o tempo. Cada caractere é processado exatamente uma vez, confirmando $O(n)$. O comportamento é o esperado pela análise teórica: $O(n + k \log k)$ com $k \log k$ desprezível para arquivos grandes.

4.2 Taxas de Compressão por Tipo de Arquivo

A taxa de compressão foi calculada com a fórmula:

$$\text{Taxa} = (1 - (\text{Tamanho Comprimido} / \text{Tamanho Original})) \times 100\%$$

Tabela 3: Taxa de compressão e entropia por tipo de arquivo.

Tipo de Arquivo	Original	Comprimido	Taxa	Entropia
Texto comum	1.898 bytes	1.074 bytes	43,40%	Baixa-média
Código-fonte (.java)	~3.500 bytes	~2.100 bytes	~40%	Média
Arquivo repetitivo	10.000 bytes	~200 bytes	~98%	Mínima
Arquivo aleatório	10.000 bytes	~12.500 bytes	-25%	Máxima

- **Texto comum (43,40%):** O português apresenta distribuição de frequências acentuadamente desequilibrada. No arquivo de teste, o espaço aparece 322 vezes e recebeu código de 3 bits; caracteres raros como 'x' e 'à' receberam 10-11 bits. Huffman reduz a média ponderada dos comprimentos dos códigos abaixo de 8 bits, resultando em compressão expressiva.
- **Código-fonte Java (~40%):** Resultado similar ao texto comum. Palavras-chave repetidas (class, public, return) contribuem para o desequilíbrio. Contudo, a presença de muitos símbolos distintos ({, }, ;, (,)) distribui as frequências, limitando o ganho.
- **Arquivo repetitivo (~98%):** Com um único símbolo dominante, a árvore tem profundidade mínima e o símbolo recebe 1 bit. Um arquivo de 10.000 bytes (80.000 bits) é representado em ~10.000 bits. Máxima redundância, entropia mínima: este é o cenário ideal para o algoritmo.
- **Arquivo aleatório (-25%):** Com distribuição uniforme entre os símbolos, a árvore fica balanceada e cada símbolo recebe aproximadamente 8 bits igual ao ASCII original. O arquivo cresce porque o cabeçalho com a tabela de frequências adiciona overhead sem compensação. Dados aleatórios têm entropia máxima de Shannon: não existe redundância a explorar, e nenhum algoritmo de compressão sem perdas consegue reduzir consistentemente esse tipo de dado.

5 Resultado Real

A Tabela 4 expõe as métricas reais capturadas durante a execução do programa com o arquivo padrão `arq_de_teste.txt`.

Tabela 4: Métricas obtidas na execução real com `arq_de_teste.txt`.

Métrica	Valor
Tamanho original	1.898 bytes (15.184 bits)
Tamanho comprimido	1.074 bytes (8.594 bits)
Taxa de compressão	43,40%
Build da árvore	13,38 ms
Compressão	16,86 ms
Tempo total	30,24 ms
Símbolos distintos	60 caracteres
Código mais curto	' ' (espaço) $\rightarrow$ 3 bits (322 ocorrências)
Código mais longo	'â' / '0' $\rightarrow$ 11 bits (1 ocorrência cada)

6 Conclusão

A implementação atingiu os objetivos propostos: compressão sem perdas com taxa de 43,40% para texto em português e desempenho de tempo linear em relação ao tamanho do arquivo, confirmando a complexidade teórica $O(n + k \log k)$.

As decisões de projeto - heap mínima manual, árvore com nós encadeados e `HashMap` para frequências - mostraram-se adequadas. A heap garantiu construção ótima da árvore, a estrutura encadeada simplificou a navegação recursiva, e o `HashMap` eliminou a limitação de 256 caracteres, tornando o sistema compatível com qualquer texto Unicode.

A análise comparativa evidenciou o princípio central do algoritmo de Huffman: a eficiência é diretamente proporcional ao desequilíbrio na distribuição de frequências. Arquivos com entropia mínima atingem compressões próximas de 98%. Por outro lado, arquivos com entropia máxima não se beneficiam e podem aumentar de tamanho pelo overhead do cabeçalho.